

Smart Coding Exam System — Complete Project Documentation

Version: 1.0.0 | **Stack:** Spring Boot + React + MySQL

Table of Contents

1. [Project Overview](#)
2. [Architecture](#)
3. [Frontend \(React + Vite + TypeScript\)](#)
 1. Entry Point & App Shell
 2. Routing & Auth
 3. Pages
 4. Components
 5. Services
 6. Types
 7. Utility Files
4. [Backend \(Spring Boot + Java\)](#)
 1. Entry Point & Config
 2. Controllers (API Layer)
 3. Services (Business Logic)
 4. Models (JPA Entities)
 5. DTOs
 6. Repositories
 7. AI/ML Integration
 8. Code Execution
 9. Security & JWT
5. [Database Schema \(MySQL\)](#)
6. [Key Workflows](#)
7. [Setup & Running](#)

1. Project Overview

The **Smart Coding Exam System** is a full-stack web application for conducting online coding examinations. It allows administrators to create exams with coding questions, and students to attempt them with real-time code execution using the Monaco Editor.

Key Features

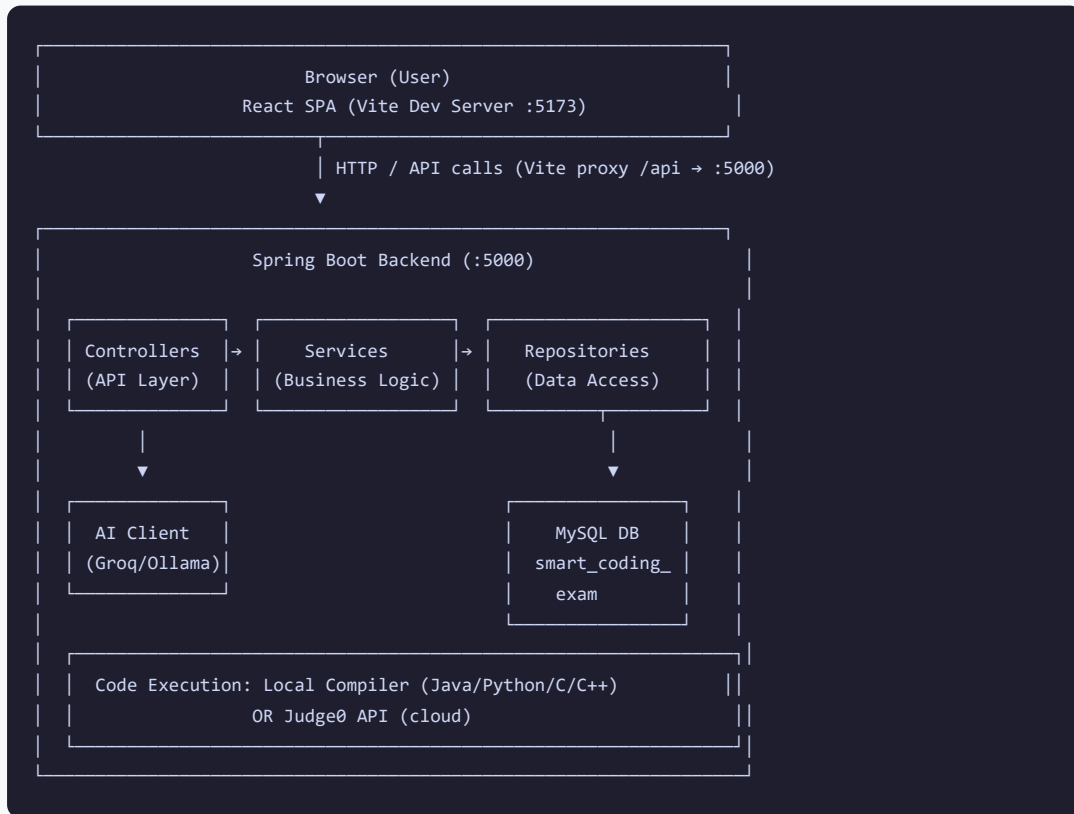
- **Role-based access:** Admin and Student roles with JWT authentication
- **Exam Management:** Create, edit, publish/unpublish exams
- **Question Management:** Manual, AI-generated, and file-upload question creation
- **Monaco Code Editor:** Industry-standard code editor with syntax highlighting for Java, Python, C, C++
- **AI Question Generation:** Generate questions via Groq (Llama 3.3) or Ollama
- **Code Execution:** Local compiler/interpreter or Judge0 API
- **Auto-grading:** Automated test case evaluation
- **Student Dashboard:** Results, leaderboards, profile management
- **Admin Dashboard:** Analytics, grade submissions, manage students

Technology Stack

Layer	Technology	Version
Frontend	React + TypeScript + Vite	React 19, Vite 7.3
Routing	wouter	3.10
UI	Bootstrap 5 + Bootstrap Icons	5.3.2
Code Editor	Monaco Editor (@monaco-editor/react)	4.7
Charts	Chart.js + react-chartjs-2	4.5 / 5.3
HTTP Client	Axios	1.7
Backend	Spring Boot	3.2.0
Database	MySQL	8.x
ORM	Hibernate (JPA)	6.3
Security	Spring Security + JWT	6.2
AI Integration	Groq API / Ollama	Custom
Code Execution	Local Compiler or Judge0	Custom

2. Architecture

System Architecture Flow



3. Frontend (React + Vite + TypeScript)

Directory: `frontend/frontend/`

3.1 Entry Point & App Shell

index.html

```
frontend/frontend/index.html
```

The HTML entry point. Loads Bootstrap 5 CSS/JS from CDN, Bootstrap Icons, and mounts the React app at

. The script `/src/main.tsx` bootstraps the React application.

main.tsx

```
frontend/frontend/src/main.tsx
```

Uses `ReactDOM.createRoot` to render the `App` component wrapped in `StrictMode`. Imports global CSS.

App.tsx

```
frontend/frontend/src/App.tsx
```

Core routing component. Uses **wouter** (lightweight React router) with `Switch` and `Route`. All pages are lazy-loaded with `React.lazy()` and wrapped in `Suspense` with a spinner fallback. The `AuthProvider` context wraps the entire app. Routes are organized by:

- **Public:** Home (/), Login (/login), Register (/register)
- **Admin:** /admin/dashboard, /admin/manage-exams, /admin/exam/add, /admin/exam/edit/:id, /admin/exam/:id/question/add, etc.
- **Student:** /student/dashboard, /exam/:id, /student/my-results, /student/profile, etc.

Protected routes enforce role-based access (`ProtectedRoute` component).

3.2 Routing & Auth

ProtectedRoute.tsx

```
frontend/frontend/src/components/auth/ProtectedRoute.tsx
```

Route guard component. Checks if user is authenticated via `AuthContext`. If not initialized, renders nothing. If not logged in, redirects to login. If user role doesn't match allowed roles, redirects to home. Otherwise renders children.

AuthContext.tsx

```
frontend/frontend/src/context/AuthContext.tsx
```

Global authentication state using React Context. Provides:

- `user` — Current user object (username, fullName, role, profilePicUrl)
- `login(user)` — Sets user state
- `logout()` — Clears tokens and user data from localStorage
- `updateUser(updates)` — Partial updates to user state
- `initialized` — Boolean flag indicating auth state is loaded

On mount, checks for existing JWT token. If found, calls `/auth/me` to validate. If not, attempts to restore user from localStorage.

3.3 Pages

HomePage

src/pages/HomePage.tsx

Landing page with gradient hero section, feature cards (Timed Exams, Monaco Editor, Analytics), and login/register CTA buttons.

LoginPage

src/pages/auth/LoginPage.tsx

Login form with username/email, password, and remember me. Uses `authService.login()` . On success, stores JWT token and user data, then redirects to appropriate dashboard based on role.

RegisterPage

src/pages/auth/RegisterPage.tsx

Registration form with username, full name, password, and confirm password. Uses `authService.register()` .

Admin Pages

File	Purpose
AdminDashboard.tsx	Stats cards (students, exams, questions, submissions) + Chart.js bar chart + Recent results table
ManageExams.tsx	List all exams, create/edit/delete/publish/unpublish actions
AddExam.tsx	Form to create new exam (title, description, duration, schedule)
EditExam.tsx	Form to edit existing exam
ManageQuestions.tsx	List questions for an exam, reorder, delete
AddQuestion.tsx	Central question creation hub with 3 tabs: Manual, AI Generate, Upload File
EditQuestion.tsx	Edit existing question with test cases
ManageStudents.tsx	List students, reset passwords, delete accounts
ViewResults.tsx	View exam results with score breakdowns
GradeSubmission.tsx	Manual grading of student submissions with code review
LeaderboardExam.tsx	Per-exam leaderboard with Chart.js

LeaderboardGlobal.tsx	Global leaderboard across all exams
-----------------------	-------------------------------------

Student Pages

File	Purpose
StudentDashboard.tsx	Upcoming exams, recent results, performance chart
ExamPage.tsx	Core exam-taking interface — Monaco Editor, timer, question navigation, code submission
MyResults.tsx	List of all exam results with scores
ResultDetail.tsx	Detailed result with per-question breakdown and code review
ReviewExam.tsx	Review submitted exam with code and test case results
ResultConfirmation.tsx	Post-submission confirmation screen with score summary
Profile.tsx	User profile editor with picture upload, password change
Leaderboard.tsx	Per-exam leaderboard for students

3.4 Components

Add Question Components

File	Purpose
ManualQuestionTab.tsx	Wrapper for QuestionForm component
QuestionForm.tsx	Complete question form with title, difficulty, marks, language selector, Monaco Editor for starter code, problem statement, advanced fields toggling, test case editor, and submit button
MonacoCodeEditor.tsx	Wrapper around @monaco-editor/react with language-specific syntax highlighting. Uses <code>key</code> prop to force re-mount on language change for proper syntax display
AdvancedFields.tsx	Collapsible section with input format, output format, constraints, sample input/output, explanation, expected solution fields
TestCaseEditor.tsx	Dynamic test case table with add/remove rows, input, expected output, weight, and sample toggle

AiGenerateTab.tsx	AI generation form with topic, language, difficulty, marks. Calls <code>aiService.generateAiQuestion()</code> and returns result via <code>onGenerated</code> callback
UploadQuestionTab.tsx	File upload with drag-and-drop, file type validation, progress indicator. Calls <code>uploadService.uploadAndGenerate()</code> with <code>examId</code> to save questions directly to backend
ExistingQuestions.tsx	Sidebar listing existing questions for the exam

UI Components

File	Purpose
Button.tsx	Reusable button with variants (primary, secondary, danger, success, outline, ghost)
Card.tsx	Reusable card with CardHeader, CardBody, CardFooter sub-components
Input.tsx	Form input with label, error message, and icon support
Modal.tsx	Bootstrap modal wrapper with header, body, footer sections
Table.tsx	Reusable table with TableHead, TableBody, TableRow, TableHeader, TableCell
Loader.tsx	Loading spinner component
EmptyState.tsx	Empty state placeholder with icon and message
Navbar.tsx	Top navigation bar with role-based links, user dropdown with profile pic and logout

3.5 Services

File	Purpose
axios.ts	Axios instance creation with base URL from env or <code>/api</code>
interceptors.ts	Request interceptor adds JWT token to headers. Response interceptor handles 401 redirect to login
api.ts	Generic HTTP helper functions (get, post, put, del) using the axios instance

authService.ts	Login, register, and get current user API calls
adminService.ts	Admin APIs: dashboard, exams CRUD, questions CRUD, students, results, grading, leaderboard
studentService.ts	Student APIs: dashboard, profile, results, leaderboard
examService.ts	Exam-taking APIs: get questions, run code, submit exam
questionService.ts	Question CRUD APIs
aiService.ts	AI question generation API
uploadService.ts	File upload with question generation API

3.6 Types

File	Key Interfaces
index.ts	ApiResponse, ExamDTO, QuestionDTO, UserDTO, ResultDTO, CodeSubmissionDTO, etc.
ai.ts	AiGenerateRequest, AiGenerateResponse (with testCases, difficulty, marks, language fields), AiGenerateResult
question.ts	QuestionFormState, TestCaseItem, DEFAULT_QUESTION_FORM, DEFAULT_TEST_CASES
upload.ts	UploadQuestionResult with savedCount, questionIds, failedCount, errors

3.7 Utility Files

File	Purpose
boilerplates.ts	Language boilerplate code templates (Java, Python, C, C++) and <code>getMonacoLanguage()</code> mapper
validators.ts	Test case validation functions
useFetch.ts	Custom React hook for API data fetching with loading/error states
index.css	Global styles (body background, dashed border utility)

4. Backend (Spring Boot + Java)

Directory: backend/src/main/java/com/example/exam/

4.1 Entry Point

OnlineExamApplication.java

Standard Spring Boot application entry point with `@SpringBootApplication` .

4.2 Configuration

File	Purpose
SecurityConfig.java	Spring Security setup: CORS (localhost:5173/5174/3000), CSRF disabled, stateless sessions, JWT filter, role-based endpoint authorization (permitAll for /api/auth/**, hasRole ADMIN for /api/admin/**, hasAnyRole STUDENT/ADMIN for /api/student/**, authenticated for /api/exam/** and /api/code/**)
AdminInitializer.java	On startup, creates a default admin user if not exists
AiQuestionProperties.java	Configuration properties for AI provider (provider, api-url, api-key, model)
MvcConfig.java	Web MVC configuration

JWT Configuration

File	Purpose
JwtTokenProvider.java	JWT token generation, validation, and username extraction using secret key from config
JwtAuthenticationFilter.java	OncePerRequestFilter that extracts JWT from Authorization header, validates, and sets SecurityContext
JwtAuthenticationEntryPoint.java	Returns 401 for unauthenticated requests

4.3 Controllers (API Layer)

File	Base Path	Key Endpoints
AuthApiController.java	/api/auth	POST /login, POST /register, GET /me
AdminApiController.java	/api/admin	GET /dashboard, GET /exams, POST /exam, PUT /exam/:id, POST /exam/:id/question, DELETE /exam/:id, POST /exam/:id/publish, POST /exam/:id/unpublish, GET /exam/:id/questions, PUT /question/:id, DELETE /question/:id, GET /students, POST /student/:id/reset-password, DELETE /student/:id, GET /results/:id, POST /results/:id/grade, GET /leaderboard/:id, GET /leaderboard/global
AiQuestionController.java	/api/ai	POST /generate — AI question generation
FileUploadController.java	/api/ai	POST /upload — Upload file, extract text, AI generates multiple questions, saves each to DB with examId
StudentApiController.java	/api/student	GET /dashboard, GET /exams, GET /exam/:id, GET /results, GET /result/:id, GET /profile, PUT /profile, POST /change-password, POST /upload-picture, GET /leaderboard/:id
ExamApiController.java	/api/exam	GET /:id/questions, POST /:id/submit, GET /results/:id
CodeExecutionController.java	/api/code	POST /run, POST /compile
ActivityLogApiController.java	/api/activity	GET /logs, POST /log

4.4 Services (Business Logic)

File	Key Responsibilities
AdminService.java	Dashboard stats, exam CRUD, question CRUD with test cases, publish/unpublish with validation, student management, grading, results. Contains inner classes: DashboardData, PublishOutcome, QuestionCreationOutcome, AdminResultsData,

	GradingPageData, DeleteQuestionOutcome, DeleteStudentOutcome, SaveGradesOutcome
AuthService.java	User authentication, registration, token generation
AiQuestionService.java	AI question generation: builds prompt, calls AI client, parses JSON response. Supports single question and multi-question parsing (for file upload). <code>generateMultipleQuestions()</code> returns List
ExamService.java	Exam-taking logic: get questions, submit exam, calculate scores
StudentService.java	Student dashboard, profile management, results, leaderboard
CodeExecutionService.java	Code execution orchestration (local or Judge0)
CodingEvaluationService.java	Test case evaluation against submitted code
LeaderboardService.java	Leaderboard calculations
FileExtractionService.java	Text extraction from uploaded files (PDF, DOCX, TXT, images)
FileStorageService.java	File storage on disk
UserService.java / UserServiceImpl.java	User CRUD operations
CustomUserDetailsService.java	Spring Security UserDetailsService implementation
ActivityLogService.java	Activity logging for audit trail

4.5 Models (JPA Entities)

Entity	Table	Key Fields
User.java	users	id, username, password (hash), email, fullName, role, profilePicUrl, mobileNumber
Exam.java	exams	id, title, description, durationInMinutes, totalMarks, published, maxViolations,

		startsAt, endsAt, createdBy (FK→User), createdAt
Question.java	questions	id, title, problemStatement, inputFormat, outputFormat, constraints, sampleInput, sampleOutput, explanation, expectedSolution, marks, difficulty, language, starterCode, executionTimeout, memoryLimit, exam (FK→Exam), testCases (OneToMany)
TestCase.java	test_cases	id, inputData, expectedOutput, weight, sample, sequenceNo, question (FK→Question)
CodingSubmission.java	coding_submissions	id, sourceCode, language, status, marksAwarded, adminRemarks, reviewed, submittedAt, exam, question, student, examResult
ExamResult.java	exam_results	id, totalScore, percentage, pass, startedAt, finishedAt, exam, student, codingSubmissions
ActivityLog.java	activity_logs	id, action, details, timestamp, user, exam

4.6 DTOs

File	Purpose
AiGenerateRequest.java	Request: topic, language, difficulty, marks
AiGenerateResponse.java	Response: title, problemStatement, inputFormat, outputFormat, constraints, sampleInput, sampleOutput, explanation, starterCode, expectedSolution, difficulty, marks, testCases list
LoginRequest.java / LoginResponse.java	Authentication DTOs
RegisterRequest.java	Registration DTO
ExamDTO.java / QuestionDTO.java / TestCaseDTO.java	Data transfer objects for exams, questions, test cases

ResultDTO.java / CodeSubmissionDTO.java	Results and submission DTOs
UserDTO.java	User data transfer object
CodeRunRequest.java / CodeCompileRequest.java / CodeExecutionResponse.java	Code execution DTOs
LeaderboardEntry.java / GlobalLeaderboardEntry.java	Leaderboard DTOs
UploadQuestionResponse.java	Upload response with savedCount, questionIds, failedCount, errors
ApiResponse.java	Generic API response wrapper

4.7 Repositories

Spring Data JPA repositories for each entity:

- UserRepository — `findByUsername()`, `countByRole()`, `findByRole()`
- ExamRepository — `getExamSubmissionCounts()`
- QuestionRepository — standard CRUD
- TestCaseRepository — standard CRUD
- CodingSubmissionRepository — `findByExam()`, `findByQuestion()`, `findByStudent()`,
`findByExamResult()`
- ExamResultRepository — `findByExamWithSubmissions()`, `findTop5ByOrderBySubmissionTimeDesc()`
- ActivityLogRepository

4.8 AI Integration

Component	Purpose
AIClient.java	HTTP client for AI providers. Supports Ollama (local) and OpenAI-compatible APIs (Groq). Handles API calls with configurable timeout, JSON request/response parsing
PromptBuilder.java	Builds structured prompts for AI. Two prompt types: <code>buildQuestionPrompt()</code> for single question generation, <code>buildMultiQuestionPrompt()</code> for parsing multiple questions from uploaded file content. Both instruct AI to return valid JSON only

AiQuestionService.java	Orchestrates AI question generation. Parses JSON responses from AI. Handles both single-object and array responses. Validates and filters out invalid question entries
------------------------	--

4.9 Code Execution

Component	Purpose
ExecutionProvider.java	Interface for code execution providers
LocalCompilerProvider.java	Executes code locally using system compilers (javac, gcc, g++, python3)
Judge0Provider.java / Judge0ClientService.java	Cloud-based code execution via Judge0 API
Judge0Properties.java	Judge0 configuration (URL, API key, timeouts)
ProviderResult.java	Execution result wrapper

4.10 Utility Classes

File	Purpose
DocxParser.java	Parse .docx files
PdfParser.java	Parse .pdf files
ImageTextExtractor.java	OCR for images
ProcessExecutionUtils.java	OS process execution utilities
SourceCodeUtils.java	Source code analysis utilities
SecurityUtils.java	Current user/authentication utilities

5. Database Schema (MySQL)

Database: smart_coding_exam | Charset: utf8mb4

Tables

users

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role VARCHAR(20) NOT NULL,          -- ROLE_ADMIN / ROLE_STUDENT  
  full_name VARCHAR(120) NOT NULL,  
  email VARCHAR(120),  
  mobile VARCHAR(20),  
  profile_picture_url VARCHAR(500),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

exams

```
CREATE TABLE exams (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  created_by_user_id BIGINT NOT NULL,  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  duration_minutes INT NOT NULL,  
  is_published BOOLEAN DEFAULT TRUE,  
  starts_at DATETIME,  
  ends_at DATETIME,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (created_by_user_id) REFERENCES users(id)  
) ENGINE=InnoDB;
```

questions

```
CREATE TABLE questions (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  exam_id BIGINT NOT NULL,  
  title VARCHAR(200) NOT NULL,  
  problem_statement TEXT NOT NULL,  
  input_format MEDIUMTEXT,  
  output_format MEDIUMTEXT,  
  constraints MEDIUMTEXT,  
  sample_input MEDIUMTEXT,  
  sample_output MEDIUMTEXT,  
  explanation MEDIUMTEXT,  
  expected_solution TEXT,  
  marks INT NOT NULL DEFAULT 10,  
  difficulty VARCHAR(20),  
  language VARCHAR(20) DEFAULT 'Java',  
  starter_code TEXT,  
  execution_timeout INT DEFAULT 5000,  
  memory_limit INT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

test_cases

```
CREATE TABLE test_cases (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  question_id BIGINT NOT NULL,  
  input_data MEDIUMTEXT NOT NULL,  
  expected_output MEDIUMTEXT NOT NULL,  
  weight INT DEFAULT 1,  
  sample BOOLEAN DEFAULT FALSE,  
  sequence_no INT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE,  
  UNIQUE KEY (question_id, sequence_no)  
) ENGINE=InnoDB;
```

coding_submissions

```
CREATE TABLE coding_submissions (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  exam_id BIGINT NOT NULL,  
  question_id BIGINT NOT NULL,  
  student_id BIGINT NOT NULL,  
  attempt_no INT DEFAULT 1,  
  language VARCHAR(30) NOT NULL,  
  source_code MEDIUMTEXT NOT NULL,  
  marks_awarded INT DEFAULT 0,  
  admin_remarks TEXT,  
  reviewed BOOLEAN DEFAULT FALSE,  
  status VARCHAR(20) DEFAULT 'SUBMITTED',  
  submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE,  
  FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE,  
  FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,  
  UNIQUE KEY (exam_id, question_id, student_id, attempt_no)  
) ENGINE=InnoDB;
```

exam_results

```
CREATE TABLE exam_results (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  exam_id BIGINT NOT NULL,  
  student_id BIGINT NOT NULL,  
  attempt_no INT DEFAULT 1,  
  total_score INT NOT NULL,  
  percentage DECIMAL(6,2) NOT NULL,  
  is_pass BOOLEAN DEFAULT FALSE,  
  started_at DATETIME,  
  finished_at DATETIME,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE,  
  FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,  
  UNIQUE KEY (exam_id, student_id, attempt_no)  
) ENGINE=InnoDB;
```

evaluation_results


```
CREATE TABLE evaluation_results (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  submission_id BIGINT NOT NULL,
  test_case_id BIGINT NOT NULL,
  pass BOOLEAN NOT NULL,
  actual_output MEDIUMTEXT,
  expected_output MEDIUMTEXT,
  execution_time_ms INT NOT NULL,
  error_message TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (submission_id) REFERENCES coding_submissions(id) ON DELETE CASCADE,
  FOREIGN KEY (test_case_id) REFERENCES test_cases(id) ON DELETE RESTRICT,
  UNIQUE KEY (submission_id, test_case_id)
) ENGINE=InnoDB;
```

attendance

```
CREATE TABLE attendance (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  exam_id BIGINT NOT NULL,
  student_id BIGINT NOT NULL,
  status VARCHAR(20) DEFAULT 'NOT_STARTED',
  last_active_at DATETIME,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE,
  FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
  UNIQUE KEY (exam_id, student_id)
) ENGINE=InnoDB;
```

activity_logs

```
CREATE TABLE activity_logs (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT,
  exam_id BIGINT,
  action VARCHAR(255) NOT NULL,
  details TEXT,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL,
  FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE SET NULL
) ENGINE=InnoDB;
```

Entity Relationship Diagram

```

users — exams (created_by)
      |
      └─ exam_results (student)
          |
          └─ coding_submissions (student)
              |
              └─ attendance (student)
                  |
                  └─ activity_logs (user)
```

```

exams — questions (exam)
      |
      |— exam_results (exam)
      |
      |— coding_submissions (exam)
      |
      |— attendance (exam)

questions — test_cases (question)
          |
          |— coding_submissions (question)

coding_submissions — evaluation_results (submission)
                  |
                  |— exam_results (exam_result)

test_cases — evaluation_results (test_case)

```

6. Key Workflows

6.1 Authentication Flow

1. User enters credentials on LoginPage → POST /api/auth/login
2. Backend validates credentials, returns JWT token + user data
3. Frontend stores token in localStorage, user data in AuthContext
4. Subsequent API calls include Authorization: Bearer header
5. JwtAuthenticationFilter validates token on every request
6. ProtectedRoute checks auth state before rendering protected pages

6.2 AI Question Generation Flow

1. Admin enters topic, language, difficulty, marks → AiGenerateTab
2. POST /api/ai/generate → AiQuestionController
3. AiQuestionService.buildQuestionPrompt() creates structured JSON prompt
4. AiClient.callApi() sends prompt to Groq/Ollama
5. AI returns JSON response parsed into AiGenerateResponse
6. Frontend populates QuestionForm with AI-generated data
7. Admin reviews and clicks "Save Question" → POST /api/admin/exam/:id/question
8. AdminService.addQuestionWithArrays() validates and saves question + test cases to DB

6.3 File Upload Multiple Questions Flow

1. Admin uploads a file (PDF/DOCX/TXT) containing 5 questions
2. POST /api/ai/upload?examId=X → FileUploadController
3. FileExtractionService.extractText() extracts text from file

4. `AiQuestionService.generateMultipleQuestions()` builds multi-question prompt and calls AI
5. AI returns JSON array of `AiGenerateResponse` objects
6. Controller iterates over each question, validates, and saves via `AdminService.addQuestionWithArrays()`
7. Returns response with `savedCount`, `questionIds`, `failedCount`, errors
8. `UploadQuestionTab` shows success/error counts to admin

6.4 Exam Taking Flow

1. Student views available exams on `StudentDashboard`
2. Clicks exam → GET `/api/student/exam/:id` → loads questions
3. `ExamPage` shows Monaco Editor for each question with timer
4. Student writes code, can run/test via POST `/api/code/run`
5. On submit → POST `/api/exam/:id/submit` → saves code + evaluates test cases
6. `ExamService` calculates score, creates `ExamResult`
7. Student redirected to `ResultConfirmation`

6.5 Grading Flow

1. Admin views exam results → GET `/api/admin/results/:id`
2. Clicks "Grade" → `GradeSubmission` page shows code + test case results
3. Admin reviews code, enters marks per question, adds remarks
4. POST `/api/admin/results/:id/grade` saves marks, updates status

7. Setup & Running

Prerequisites

- Node.js 18+
- Java 17+ (Maven wrapper included)
- MySQL 8+
- For local code execution: JDK, Python, GCC/G++ installed
- For AI: Groq API key (free) or Ollama running locally

Database Setup

```
mysql -u root -p < database/schema_mysql.sql
mysql -u root -p smart_coding_exam < database/clean_data.sql
```

Backend Setup

```
cd backend
# Configure application.properties:
#   - MySQL credentials
#   - AI provider (Groq/Ollama) with API key
#   - JWT secret
./mvnw spring-boot:run
```

Frontend Setup

```
cd frontend/frontend
npm install
npm run dev
```

Access

- Frontend: http://localhost:5173
- Backend: http://localhost:5000
- Default admin: username admin , password from DB

Environment Variables

Variable	Default	Description
MYSQL_URL	jdbc:mysql://localhost:3306/smart_coding_exam	Database URL
MYSQL_USER	root	Database user
MYSQL_PASSWORD	—	Database password
JWT_SECRET	Base64 secret	JWT signing key
AI_PROVIDER	openai	AI provider (openai/ollama)
AI_API_URL	https://api.groq.com/openai/v1/chat/completions	AI API endpoint
AI_API_KEY	—	AI API key (Groq)
AI_MODEL	llama-3.3-70b-versatile	AI model name

 **Note:** This documentation covers all files and their purposes in the Smart Coding Exam System. The project follows a clean architecture pattern with separation of concerns: Controllers (API) → Services (Business Logic) → Repositories (Data Access) → Database.

Generated on: August 2026 | Smart Coding Exam System v1.0.0